

Report
(Special Assignment)

2EC202CC23 – FPGA-based system Design

Submitted By:

Dhruvi Thakkar (24BEC160)

Vidhi Patel(24BEC158)



Semester IV

B.Tech Electronics and Communication Engineering

School of Engineering

Institute of Technology

Nirma University

Ahmedabad 382481

Topic: Four-Floor Dual Elevator System

Abstract

Modern buildings require efficient elevator control systems to reduce waiting time and improve passenger convenience. This project presents a 4-floor dual-elevator control system implemented in Verilog HDL and FPGA technology. The system accepts a binary floor request and determines which elevator is closest to the requested floor. Based on the calculated distance, the controller automatically selects the most appropriate elevator. The selected elevator moves step by step toward the requested floor using clock-controlled sequential logic. Direction indicators display whether the elevator is moving up or down, and seven-segment displays show the current floor of each elevator. The design demonstrates how FPGA-based digital systems can efficiently control real-world applications such as elevator management.

Keywords

Elevator Control System, FPGA, Verilog HDL, Dual Elevator System, Digital Design, Distance Calculation, Seven Segment Display

Literature Survey / State of the Art Technology Available

Elevator systems are widely used in modern buildings, and their efficiency directly affects user convenience. Traditional elevator control systems typically use simple scheduling algorithms such as first-come, first-served. Recent advancements in digital design allow FPGA-based elevator systems, which offer higher flexibility and programmability. FPGA implementations enable designers to modify control logic easily and experiment with different scheduling algorithms. Research implementations frequently demonstrate elevator systems using finite state machines and sequential logic designed in Verilog or VHDL.

Limitations or Drawbacks of Currently Available Technology

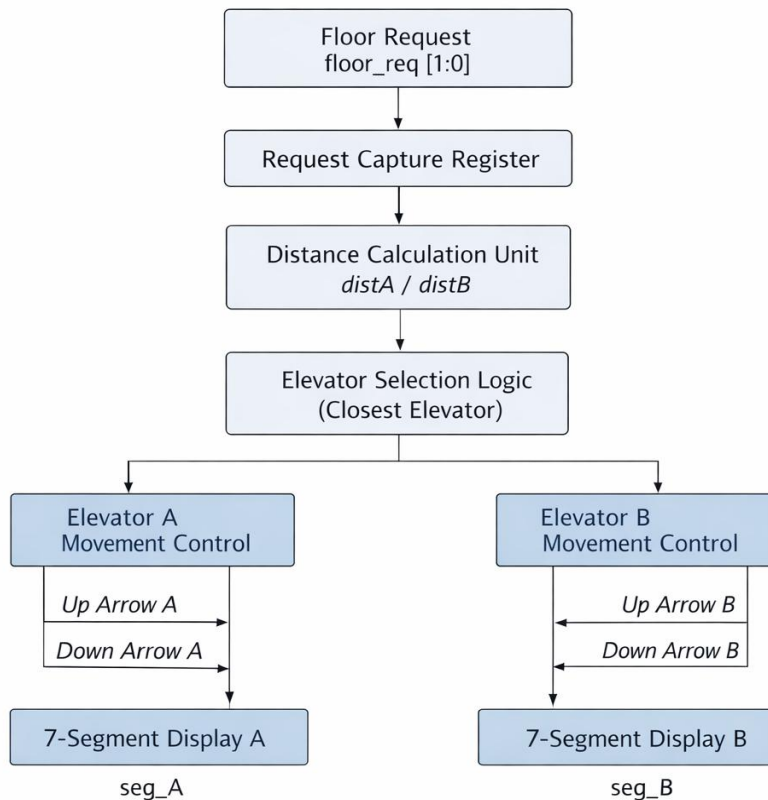
- Traditional elevator systems often rely on fixed scheduling algorithms.
- Requests may be processed sequentially, increasing waiting time.

- Some systems lack clear visual indicators for floor and direction.
- Hardware-based systems are difficult to modify when adding more floors.
- Inefficient scheduling may cause unnecessary elevator movement and energy usage.

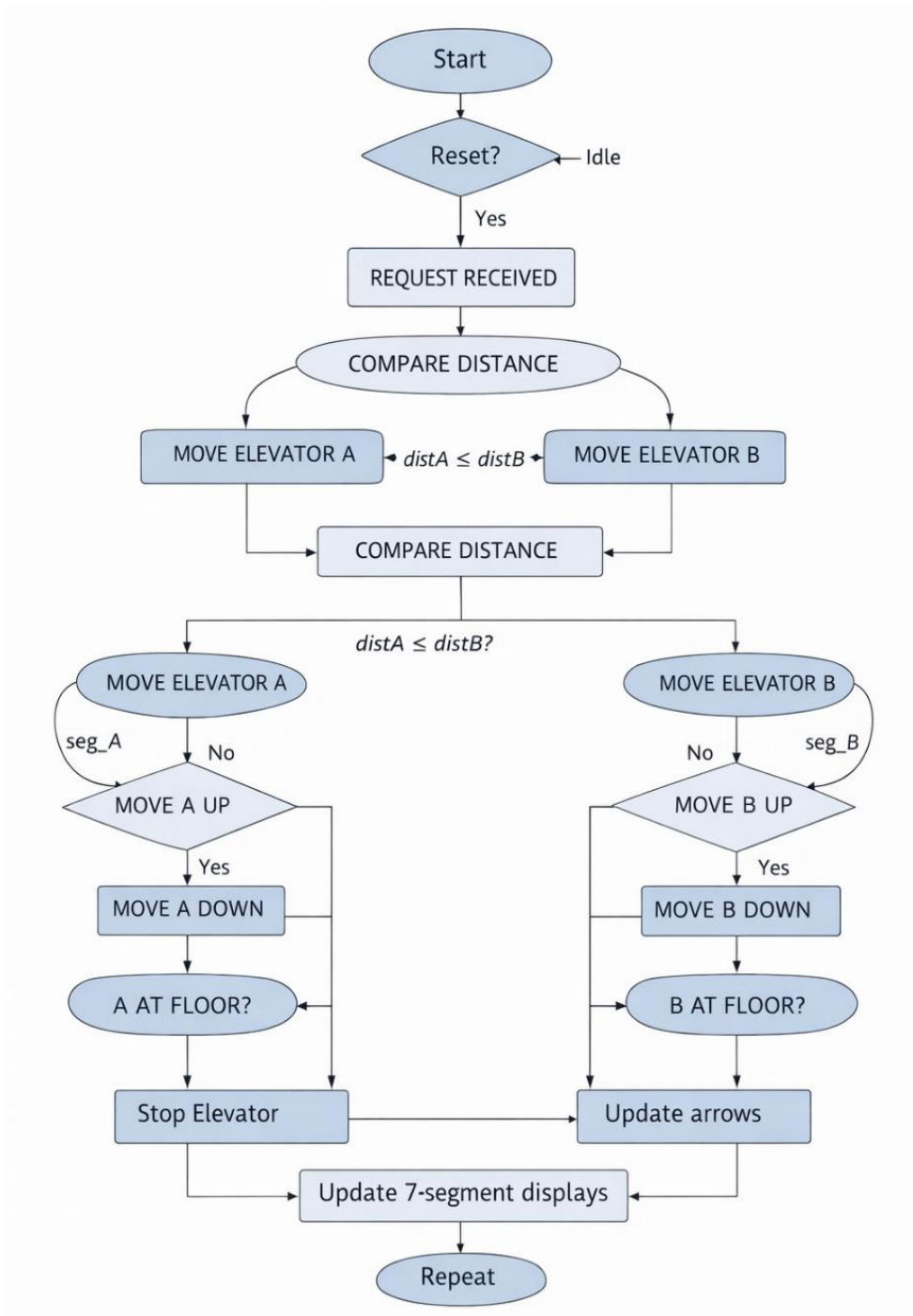
Proposed Solution / Methodology

The proposed system implements a dual elevator control system using Verilog HDL on an FPGA platform. The system receives a binary floor request input representing floors 0 to 3. The controller calculates the distance between each elevator and the requested floor and selects the closest elevator. The selected elevator moves one floor per clock cycle until the requested floor is reached. Direction indicators show the movement direction, while seven-segment displays present the current floor of each elevator. The modular FPGA design allows flexibility, scalability, and efficient system operation.

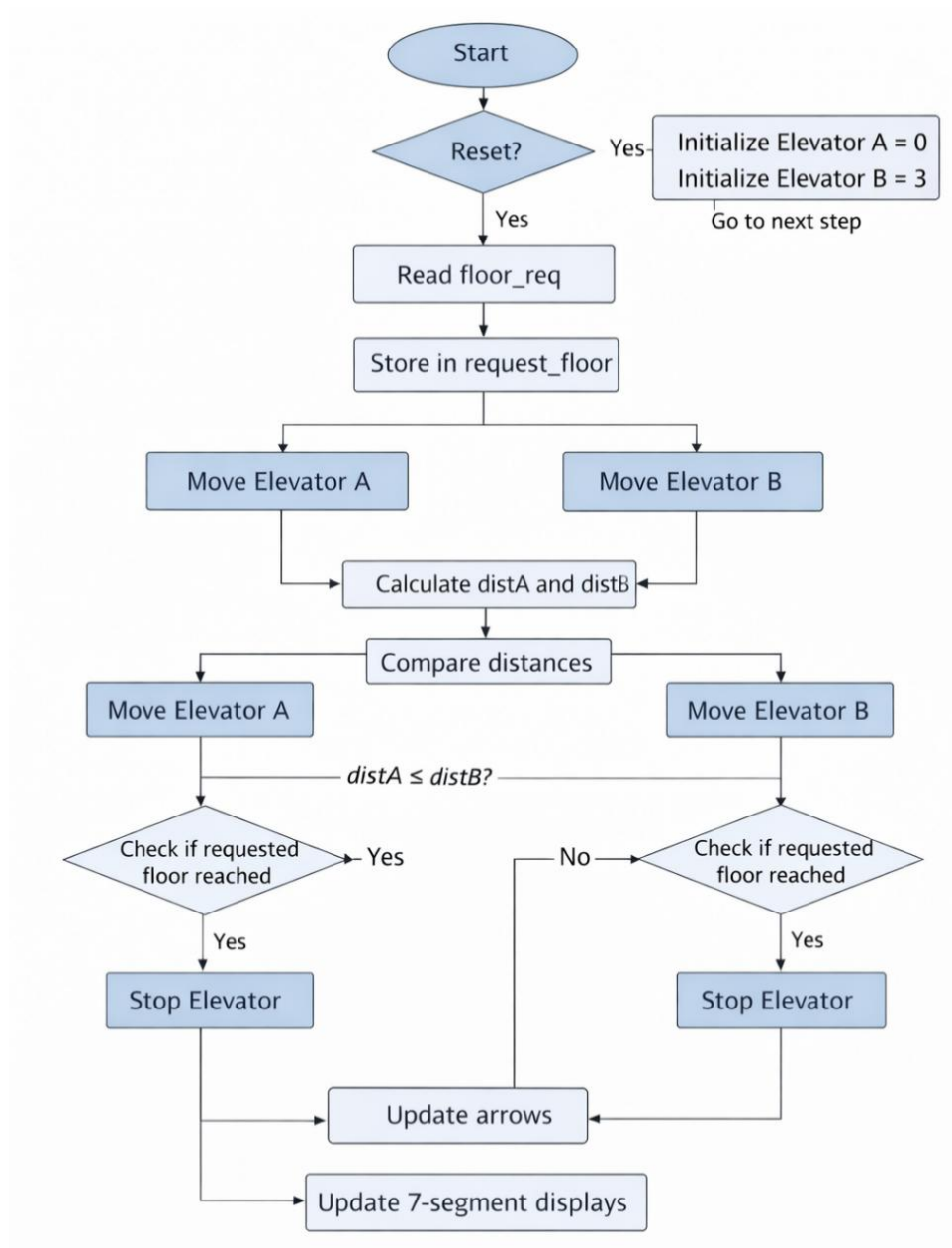
Block Diagram



State Diagram

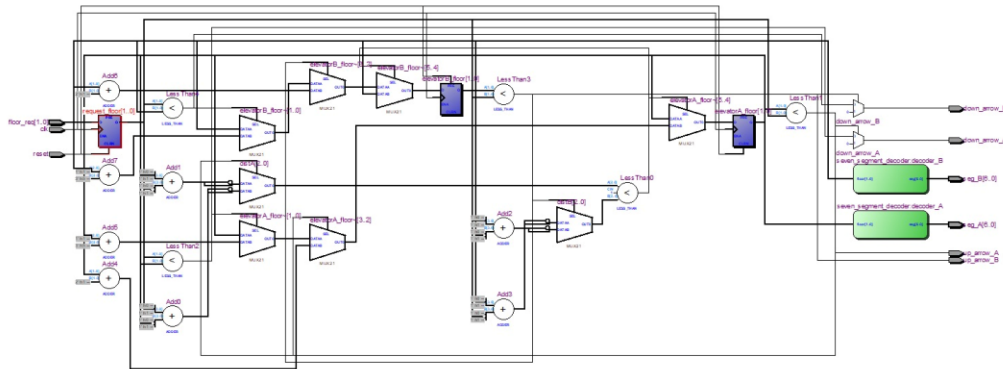


Flow Chart of the Code

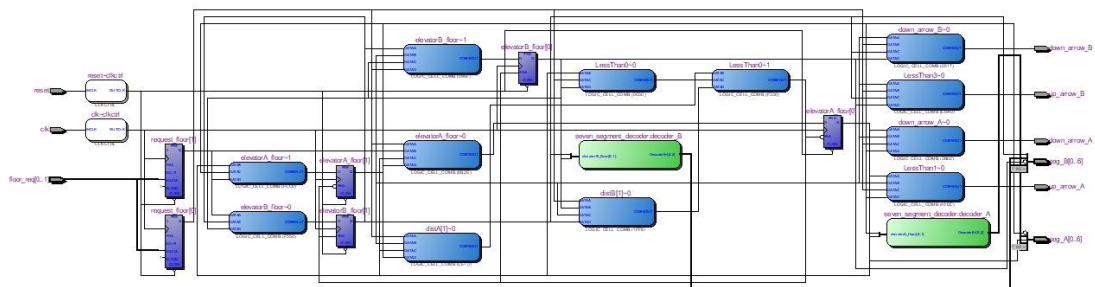


Results in Terms of RTL, TTL and Waveforms

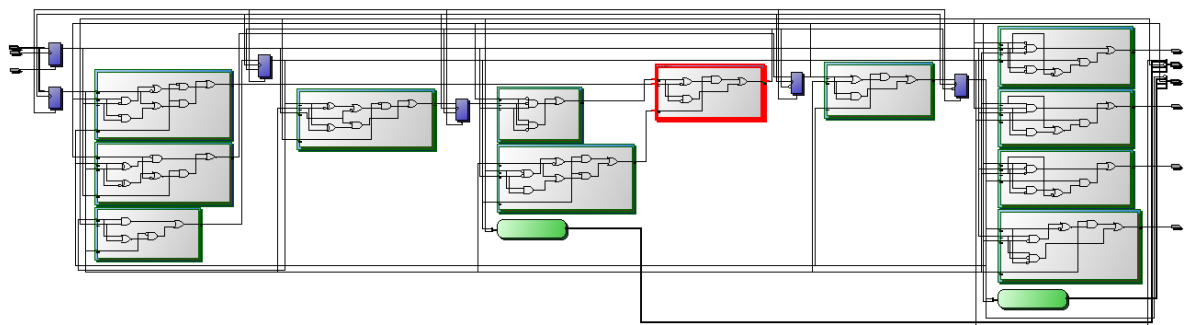
RTL view



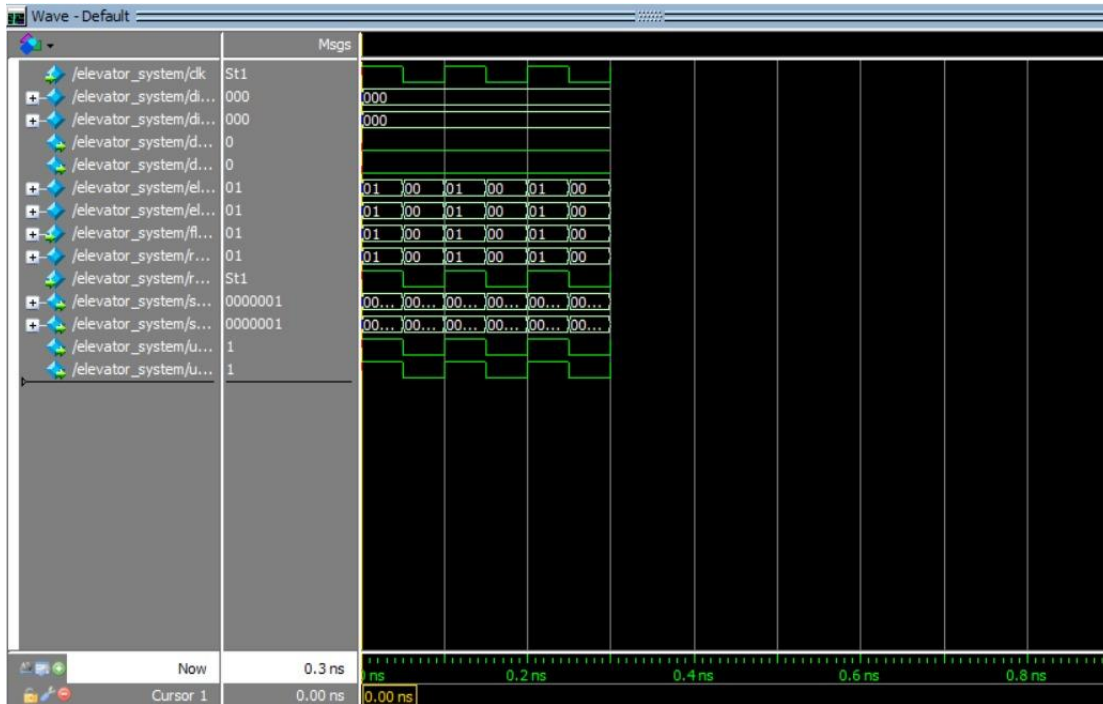
TTL view



TTL view (Expanded)



ModelSim Simulation



Conclusion

The implemented system demonstrates a dual elevator control mechanism using FPGA and Verilog HDL. The controller efficiently selects the closest elevator based on distance calculations and controls its movement toward the requested floor. Direction indicators and seven-segment displays provide clear feedback regarding elevator operation. The modular design allows the system to be expanded for additional floors or elevators.

References

- M. Morris Mano – Digital Design
- Harris & Harris – Digital Design and Computer Architecture

Code in the Appendix

```
module Elevator_system(
    input clk,        // Manual clock
    input reset,      // Reset
    input [1:0] floor_req, // Binary floor request (00–11)
    output [6:0] seg_A, // Elevator A display
    output [6:0] seg_B, // Elevator B display
    output reg up_arrow_A,
    output reg down_arrow_A,
    output reg up_arrow_B,
    output reg down_arrow_B
);
// —— Elevator Floor Registers ——
reg [1:0] elevatorA_floor;
reg [1:0] elevatorB_floor;
// —— Requested Floor ——
reg [1:0] request_floor;
// —— Request Capture ——
always @(posedge clk or posedge reset)
begin
    if (reset)
        request_floor <= 2'b00;
    else
        request_floor <= floor_req;
end
// —— Distance Calculation ——
wire [2:0] distA;
wire [2:0] distB;
assign distA = (request_floor > elevatorA_floor) ?
    ({1'b0, request_floor} - {1'b0, elevatorA_floor}) :
```

```

        ({1'b0, elevatorA_floor} - {1'b0, request_floor});
assign distB = (request_floor > elevatorB_floor) ?
        ({1'b0, request_floor} - {1'b0, elevatorB_floor}) :
        ({1'b0, elevatorB_floor} - {1'b0, request_floor});
// ----- Elevator Movement -----
always @(posedge clk or posedge reset)
begin
    if (reset)
    begin
        elevatorA_floor <= 2'b00; // Floor 0
        elevatorB_floor <= 2'b11; // Floor 3
    end
    else
    begin
        if (distA <= distB)
        begin
            if (request_floor > elevatorA_floor)
                elevatorA_floor <= elevatorA_floor + 2'b01;
            else if (request_floor < elevatorA_floor)
                elevatorA_floor <= elevatorA_floor - 2'b01;
        end
        else
        begin
            if (request_floor > elevatorB_floor)
                elevatorB_floor <= elevatorB_floor + 2'b01;
            else if (request_floor < elevatorB_floor)
                elevatorB_floor <= elevatorB_floor - 2'b01;
        end
    end
end
// ----- Arrow Logic Elevator A -----

```

```

always @(*)
begin
    if (request_floor > elevatorA_floor)
    begin
        up_arrow_A = 1'b1;
        down_arrow_A = 1'b0;
    end
    else if (request_floor < elevatorA_floor)
    begin
        up_arrow_A = 1'b0;
        down_arrow_A = 1'b1;
    end
    else
    begin
        up_arrow_A = 1'b0;
        down_arrow_A = 1'b0;
    end
end

```

// ——— Arrow Logic Elevator B —————

```

always @(*)
begin
    if (request_floor > elevatorB_floor)
    begin
        up_arrow_B = 1'b1;
        down_arrow_B = 1'b0;
    end
    else if (request_floor < elevatorB_floor)
    begin
        up_arrow_B = 1'b0;
        down_arrow_B = 1'b1;
    end
end

```

```

    else
    begin
        up_arrow_B = 1'b0;
        down_arrow_B = 1'b0;
    end
end

// ----- 7-Segment Display Modules -----
seven_segment_decoder decoder_A (
    .floor(elevatorA_floor),
    .seg(seg_A)
);
seven_segment_decoder decoder_B (
    .floor(elevatorB_floor),
    .seg(seg_B)
);
endmodule

// ----- 7-Segment Decoder -----
module seven_segment_decoder(
    input [1:0] floor,
    output reg [6:0] seg);
always @(*)
begin
    case(floor)
        2'b00: seg = 7'b1000000; // 0
        2'b01: seg = 7'b1111001; // 1
        2'b10: seg = 7'b0100100; // 2
        2'b11: seg = 7'b0110000; // 3
        default: seg = 7'b1111111;
    endcase
end
endmodule

```